

FROM GAMER TO GAME DEV

For gamers starting from zero who want a programming portfolio, not tutorial clones. Build fundamentals in order, then go deep on the game systems you actually care about. Passion matters because depth is what employers notice.

PORTFOLIO / TECH DEMO ROADMAP

DEPTH BEATS VOLUME

Employers notice depth, not shallow project lists.

1 SETUP

TECH

Visual Studio Community, .NET SDK, C#, debugger, terminal.

BUILD

Run Hello World, change it, hit a breakpoint, and explain what changed.

2 CONSOLE GAMES

TECH

C# console apps, variables, loops, functions, lists, classes, file save/load.

BUILD

Make number guess, text combat, shop menu, inventory, and save-file exercises.

3 2D RETRO GAMES

TECH

C# with raylib-cs or MonoGame, game loop, sprites, input, collision, score.

BUILD

Build Pong, Breakout, Asteroids-style movement, restart flow, and a clean game over.

4 GAMEPLAY PROGRAMMING

TECH

State machines, events, timers, cameras, animation states, enemy AI, UI flow.

BUILD

Prototype one mechanic: abilities, enemy states, dialogue, inventory, or combat.

5 SYSTEMS PROGRAMMING

TECH

Data structures, object pools, spatial grids, save formats, debug views, profiling.

BUILD

Build one reusable system with tests, debug overlay, edge cases, and a short write-up.

6 3D RETRO GAMES

TECH

Unity or Godot, 3D transforms, cameras, physics, materials, lighting, builds.

BUILD

Make a small 3D retro demo: maze, arena, space shooter, kart, or puzzle room.

7. CHOOSE YOUR OWN TECH DEMO PATH

Pick the path that matches your taste. Build fewer demos, but go deeper: playable build, readable code, debug tools, tradeoffs, and a walkthrough.

AI + ENEMY BEHAVIOUR

State machines, steering, pathfinding, squad logic, combat decisions.

NETWORKING

Client prediction, rollback, lobbies, sync bugs, multiplayer debug views.

GRAPHICS PROGRAMMING

Shaders, materials, lighting, render passes, post effects, visual performance.

TOOLS PROGRAMMING

Level editors, debug inspectors, content pipelines, designer-friendly workflows.

SIMULATION + OPTIMIZATION

Crowds, economies, projectiles, large object counts, profiling, performance budgets.

PROCEDURAL GENERATION

Maps, loot, quests, biomes, encounter rules, seeded worlds.

RPG / STRATEGY SYSTEMS

Inventory, abilities, quests, economy, tech trees, save/load.

UI / AUDIO / FEEL

Menus, accessibility, controller UX, sound, hit stop, juice, responsive feedback.

TURN DEPTH INTO CAREER PROOF

8 PORTFOLIO / RESUME / APPLICATION PREP

PREP

Project pages, resume bullets, demo videos, screenshots, short write-ups, and application materials.

PROOF

Show the best 1-3 demos clearly: what problem you solved, what tradeoffs you made, and what code is worth reviewing.

9 APPLY + INTERVIEW PRACTICE

PREP

Targeted applications, role tracking, follow-ups, recruiter screens, and project walkthrough rehearsal.

PROOF

Practise responding clearly, explaining your own projects, defending decisions, and telling concise stories from the work.